

ATM - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Insert card -> PIN -> withdraw -> machine gives notes -> card comes out.

Story - how this LLD works

An ATM is like a vending machine for cash, but it must follow steps in order. You cannot withdraw before entering PIN. You cannot insert two cards at once. So we model the machine as states: Idle, Card Inserted, Authenticated, Dispense Cash. Giving notes is a chain: try Rs.2000 notes first, then Rs.500, then Rs.100.

Who does what

ATMMachine: The face of the ATM. You talk to it; it forwards work to the current state.

IdleState: No card. Only insertCard is allowed.

CardInsertedState: Waiting for PIN. Wrong PIN? Stay here. Right PIN? Move to Authenticated.

AuthenticatedState: Logged in. Choose withdraw (demo).

DispenseCashState: Check balances, run note chain, give cash, eject card, back to Idle.

CashDispenser chain: 2000 handler -> 500 handler -> 100 handler. Each takes what it can, passes rest.

Card / Account: Card has PIN; Account has balance.

Main pieces

ATMMachine

- current state
- current card
- insert/pin/withdraw

States

- Idle
- CardInserted
- Authenticated
- DispenseCash

Dispenser chain

- 2000 -> 500 -> 100
- canDispense / dispense

Account

- balance
- deduct on success

Step-by-step at runtime

1. Machine starts in Idle.
2. insertCard(card) -> now CardInserted.
3. enterPin: if wrong, stay; if correct -> Authenticated.
4. selectOption(WITHDRAW) -> DispenseCash.
5. dispenseCash(amount): check ATM has enough cash, account has enough balance.
6. Chain asks: can we make this amount with available notes?
7. If yes, notes are deducted, account balance reduced, card ejected -> Idle.
8. If no, message shown, card ejected -> Idle (no money given).

Why this design (plain words)

Why State? - Each step allows different actions. Avoids one giant if status==... mess.

Why Chain of Responsibility? - Each denomination is one small class. Add Rs.50 later by adding one handler.

Why Factory for state? - If ATM restarts mid-session, rebuild the right state from saved status.

Remember

- Wrong step in wrong state is rejected (e.g. withdraw while Idle).
- Dispensing is greedy (max big notes first) - interview follow-up topic.
- Demo option string is mostly ignored - always goes to withdraw.

OOP in this project

Encapsulation - Only the current state decides what is allowed.

Abstraction - ATMState defines the buttons; each state implements them differently.

Inheritance - IdleState etc. are ATMStates; note classes are CashDispensers.

Polymorphism - machine.insertCard() calls whatever state is current.

DESIGN.md - full file

Complete markdown from lld/atm/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing an ATM - LLD Q&A

Run: `python -m lld.atm.main`

Patterns: State · Chain of Responsibility · Factory · Repository

Q1. What problem are we solving?

A. Design an ATM cash-withdrawal machine that:

Accepts a card linked to an account

Authenticates with a PIN

Allows withdrawal

Dispenses cash using available note denominations (Rs.2000, Rs.500, Rs.100)

Validates ATM cash, account balance, and denomination feasibility

Deducts from ATM inventory and account, then ejects the card

Invalid actions in the wrong state must be rejected (e.g. withdraw before PIN).

Q2. What are the core entities?

- Entity - Responsibility
- ATM - Machine id, note counts, cash available, status
- Card - Card number, PIN, linked Account
- Account - Account number, balance
- ATMMachine - Context - holds current card + state; user-facing API
- ATMState - Abstract state - defines valid operations
- CashDispenser - CoR handler for one denomination

Q3. How is the package structured?

```

atm/
|-- model/      # ATM, Card, Account
|-- state/      # Idle -> CardInserted -> Authenticated -> DispenseCash
|-- cor/        # Cash dispenser chain (Rs.2000 -> Rs.500 -> Rs.100)
|-- service/    # ATMMachine (facade/context)
|-- factory/    # ATMStateFactory (status -> state)
|-- repository/ # In-memory ATM storage
|-- enums/     # ATMStatus

```

Q4. Why State Pattern?

A. ATM behavior depends on lifecycle stage. Each state allows only valid operations and controls transitions.

IDLE -> CARD_INSERTED -> AUTHENTICATED -> DISPENSE_CASH -> IDLE

- State - Allows
- IdleState - Insert card
- CardInsertedState - Enter PIN / eject
- AuthenticatedState - Select option (withdraw) / eject
- DispenseCashState - Dispense cash -> eject -> IDLE
- ATMMachine delegates every action to self.state - no giant if status == ... switch in one class.

Q5. Why Chain of Responsibility for cash?

A. Each denomination handler tries to use its notes, then passes the remainder to the next:

Rs.2000 Dispenser -> Rs.500 Dispenser -> Rs.100 Dispenser

`can_dispense(atm, amount)` - check if a valid combination exists

`dispense(atm, amount)` - deduct notes and print breakdown

Easy to add a new denomination: new handler + wire it in `CashDispenserChainBuilder`.

Q6. What is the happy-path withdrawal flow?

```

insert_card(card)  -> Idle -> CardInserted
enter_pin(pin)     -> if correct -> Authenticated
select_option("WITHDRAW") -> DispenseCash
dispense_cash(amount)
-> check ATM cash >= amount
-> check account balance >= amount
-> chain.can_dispense(...)
-> chain.dispense(...)
-> deduct account + ATM cash
-> eject_card -> Idle

```

Q7. What are the failure paths?

- Condition - Behavior
- Action without card - Rejected in Idle

- Wrong PIN - Stay in CardInserted
- Amount > ATM cash - Eject -> Idle
- Amount > balance - Eject -> Idle
- No valid note combo - Cannot dispense -> eject -> Idle

Q8. Why Factory for states?

A. ATMStateFactory.get_state(status, machine) rebuilds the correct state object from a persisted ATMStatus. Useful after restart / crash recovery.

Q9. How do you extend this design?

- Add... - How
- Deposit / balance inquiry - New states or branches in AuthenticatedState.select_option
- New note (e.g. Rs.50) - New CashDispenser + chain builder
- PIN retry lockout - Counter in CardInsertedState
- Real DB - Swap ATMRepository implementation

Q10. Common interview follow-ups

Q. Why not one class with a status enum and if-else?

State Pattern keeps each state's rules local and open for new states without bloating one method.

Q. Is the CoR greedy?

Yes - max higher notes first. It may fail when a different mix would work. Discuss DP/backtracking as an alternative.

Q. What does select_option ignore?

In this demo it always goes to withdrawal regardless of the option string - intentional simplification.

Source code - lld/atm/

18 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (31 lines)

```
1| from .model.account import Account
2| from .model.atm import ATM
3| from .model.card import Card
4| from .repository.atm_repository import ATMRepository
5| from .service.atm_machine import ATMMachine
6|
7|
8| def main() -> None:
9|     card = Card(
10|         "CARD123",
11|         "1234",
12|         Account("ACC123", 5000),
13|     )
14|
15|     atm1 = ATM("ATM1", 5, 5, 20)
16|     atm2 = ATM("ATM2", 0, 2, 5)
17|
18|     atm_repository = ATMRepository()
19|     atm_repository.save(atm1)
20|     atm_repository.save(atm2)
21|
22|     atm_machine2 = ATMMachine("ATM2", atm_repository)
23|
24|     atm_machine2.insert_card(card)
25|     atm_machine2.enter_pin("1234")
26|     atm_machine2.select_option("WITHDRAW")
27|     atm_machine2.dispense_cash(1410)
28|
29|
30| if __name__ == "__main__":
31|     main()
```

FILE: service/atm_machine.py (40 lines)

```
1| from __future__ import annotations
2|
3| from typing import Optional
4|
5| from ..factory.atm_state_factory import ATMStateFactory
6| from ..model.atm import ATM
7| from ..model.card import Card
8| from ..repository.atm_repository import ATMRepository
9| from ..state.atm_state import ATMState
10|
11|
12| class ATMMachine:
13|     def __init__(self, atm_id: str, atm_repository: ATMRepository):
14|         self.atm_repository = atm_repository
15|         atm = atm_repository.get_by_id(atm_id)
16|         if atm is None:
17|             raise RuntimeError("ATM not found")
18|         self.atm: ATM = atm
19|         self.state: ATMState = ATMStateFactory.get_state(self.atm.status, self)
20|         self.current_card: Optional[Card] = None
21|
22|     def insert_card(self, card: Card) -> None:
23|         self.state.insert_card(card)
24|
25|     def enter_pin(self, pin: str) -> None:
26|         self.state.enter_pin(pin)
27|
28|     def select_option(self, option: str) -> None:
29|         self.state.select_option(option)
30|
31|     def dispense_cash(self, amount: int) -> None:
32|         self.state.dispense_cash(amount)
33|
34|     def eject_card(self) -> None:
35|         self.state.eject_card()
36|
37|     def set_state(self, state: ATMState) -> None:
38|         self.state = state
39|         self.atm.status = state.get_status()
40|         # persist the changes in db
```

FILE: model/account.py (7 lines)

```
1| from dataclasses import dataclass
2|
3|
4| @dataclass
5| class Account:
6|     account_number: str
7|     balance: float
```

FILE: model/atm.py (31 lines)

(cont.) model/atm.py

```

1| from dataclasses import dataclass
2|
3| from ..enums.atm_status import ATMStatus
4|
5|
6| @dataclass
7| class ATM:
8|     id: str
9|     status: ATMStatus = ATMStatus.IDLE
10|     cash_available: float = 0.0
11|     two_thousand_count: int = 0
12|     five_hundred_count: int = 0
13|     one_hundred_count: int = 0
14|
15|     def __init__(
16|         self,
17|         id: str,
18|         two_thousand_count: int,
19|         five_hundred_count: int,
20|         one_hundred_count: int,
21|     ):
22|         self.id = id
23|         self.cash_available = (
24|             2000 * two_thousand_count
25|             + 500 * five_hundred_count
26|             + 100 * one_hundred_count
27|         )
28|         self.status = ATMStatus.IDLE
29|         self.two_thousand_count = two_thousand_count
30|         self.five_hundred_count = five_hundred_count
31|         self.one_hundred_count = one_hundred_count

```

FILE: model/card.py (10 lines)

```

1| from dataclasses import dataclass
2|
3| from .account import Account
4|
5|
6| @dataclass
7| class Card:
8|     card_number: str
9|     pin: str
10|     account: Account

```

FILE: cor/cash_dispenser.py (17 lines)

```

1| from abc import ABC, abstractmethod
2|
3| from ..model.atm import ATM
4|
5|
6| class CashDispenser(ABC):
7|     @abstractmethod
8|     def set_next_dispenser(self, next_dispenser: "CashDispenser") -> None:
9|         pass
10|
11|     @abstractmethod
12|     def can_dispense(self, atm: ATM, amount: int) -> bool:
13|         pass
14|
15|     @abstractmethod
16|     def dispense(self, atm: ATM, amount: int) -> None:
17|         pass

```

FILE: cor/cash_dispenser_chain_builder.py (16 lines)

```

1| from .cash_dispenser import CashDispenser
2| from .five_hundred_dispenser import FiveHundredDispenser
3| from .one_hundred_dispenser import OneHundredDispenser
4| from .two_thousand_dispenser import TwoThousandDispenser
5|
6|
7| class CashDispenserChainBuilder:
8|     @staticmethod
9|     def build_chain() -> CashDispenser:
10|         d1 = TwoThousandDispenser()
11|         d2 = FiveHundredDispenser()
12|         d3 = OneHundredDispenser()
13|
14|         d1.set_next_dispenser(d2)
15|         d2.set_next_dispenser(d3)
16|         return d1

```

FILE: cor/five_hundred_dispenser.py (30 lines)

```

1| from ..model.atm import ATM
2| from .cash_dispenser import CashDispenser
3|
4|
5| class FiveHundredDispenser(CashDispenser):
6|     def __init__(self):
7|         self._next: CashDispenser | None = None
8|
9|     def set_next_dispenser(self, next_dispenser: CashDispenser) -> None:
10|         self._next = next_dispenser
11|
12|     def can_dispense(self, atm: ATM, amount: int) -> bool:
13|         available_notes = atm.five_hundred_count
14|         notes = min(amount // 500, available_notes)
15|         remainder = amount - notes * 500
16|         return remainder == 0 or (

```

```

17|         self._next is not None and self._next.can_dispende(atm, remainder)
18|     )
19|
20| def dispense(self, atm: ATM, amount: int) -> None:
21|     available_notes = atm.five_hundred_count
22|     notes = min(amount // 500, available_notes)
23|     atm.five_hundred_count = available_notes - notes
24|     remainder = amount - notes * 500
25|
26|     if notes > 0:
27|         print(f"Dispensed {notes} x Rs.500 notes")
28|
29|     if remainder > 0 and self._next is not None:
30|         self._next.dispense(atm, remainder)

```

FILE: cor/one_hundred_dispenser.py (23 lines)

```

1| from ..model.atm import ATM
2| from .cash_dispenser import CashDispenser
3|
4|
5| class OneHundredDispenser(CashDispenser):
6|     def __init__(self):
7|         self._next: CashDispenser | None = None
8|
9|     def set_next_dispenser(self, next_dispenser: CashDispenser) -> None:
10|         self._next = next_dispenser
11|
12|     def can_dispende(self, atm: ATM, amount: int) -> bool:
13|         available_notes = atm.one_hundred_count
14|         notes = min(amount // 100, available_notes)
15|         remainder = amount - notes * 100
16|         return remainder == 0
17|
18|     def dispense(self, atm: ATM, amount: int) -> None:
19|         available_notes = atm.one_hundred_count
20|         notes = min(amount // 100, available_notes)
21|         atm.one_hundred_count = available_notes - notes
22|         if notes > 0:
23|             print(f"Dispensed {notes} x Rs.100 notes")

```

FILE: cor/two_thousand_dispenser.py (30 lines)

```

1| from ..model.atm import ATM
2| from .cash_dispenser import CashDispenser
3|
4|
5| class TwoThousandDispenser(CashDispenser):
6|     def __init__(self):
7|         self._next: CashDispenser | None = None
8|
9|     def set_next_dispenser(self, next_dispenser: CashDispenser) -> None:
10|         self._next = next_dispenser
11|
12|     def can_dispende(self, atm: ATM, amount: int) -> bool:
13|         count = atm.two_thousand_count
14|         notes = min(amount // 2000, count)
15|         remainder = amount - notes * 2000
16|         return remainder == 0 or (
17|             self._next is not None and self._next.can_dispende(atm, remainder)
18|         )
19|
20|     def dispense(self, atm: ATM, amount: int) -> None:
21|         count = atm.two_thousand_count
22|         notes = min(amount // 2000, count)
23|         atm.two_thousand_count = count - notes
24|
25|         remainder = amount - notes * 2000
26|         if notes > 0:
27|             print(f"Dispensed {notes} x 2000 notes")
28|
29|         if remainder > 0 and self._next is not None:
30|             self._next.dispense(atm, remainder)

```

FILE: state/atm_state.py (30 lines)

```

1| from abc import ABC, abstractmethod
2|
3| from ..enums.atm_status import ATMStatus
4| from ..model.card import Card
5|
6|
7| class ATMState(ABC):
8|     @abstractmethod
9|     def insert_card(self, card: Card) -> None:
10|         pass
11|
12|     @abstractmethod
13|     def enter_pin(self, pin: str) -> None:
14|         pass
15|
16|     @abstractmethod
17|     def select_option(self, option: str) -> None:
18|         pass
19|
20|     @abstractmethod
21|     def dispense_cash(self, amount: int) -> None:
22|         pass
23|
24|     @abstractmethod
25|     def eject_card(self) -> None:
26|         pass

```

```

27|
28| @abstractmethod
29| def get_status(self) -> ATMStatus:
30|     pass

```

FILE: state/authenticated_state.py (41 lines)

```

1| from __future__ import annotations
2|
3| from typing import TYPE_CHECKING
4|
5| from ..enums.atm_status import ATMStatus
6| from ..model.card import Card
7| from .atm_state import ATMState
8|
9| if TYPE_CHECKING:
10|     from ..service.atm_machine import ATMMachine
11|
12|
13| class AuthenticatedState(ATMState):
14|     def __init__(self, atm_machine: ATMMachine):
15|         self._atm_machine = atm_machine
16|
17|     def insert_card(self, card: Card) -> None:
18|         print("Card already inserted.")
19|
20|     def enter_pin(self, pin: str) -> None:
21|         print("Already authenticated.")
22|
23|     def select_option(self, option: str) -> None:
24|         from .dispense_cash_state import DispenseCashState
25|
26|         # can add options like deposit, check balance based on option selected.
27|         print("Option selected: Withdrawal.")
28|         self._atm_machine.set_state(DispenseCashState(self._atm_machine))
29|
30|     def dispense_cash(self, amount: int) -> None:
31|         print("Select an option first.")
32|
33|     def eject_card(self) -> None:
34|         from .idle_state import IdleState
35|
36|         self._atm_machine.current_card = None
37|         print("Card ejected.")
38|         self._atm_machine.set_state(IdleState(self._atm_machine))
39|
40|     def get_status(self) -> ATMStatus:
41|         return ATMStatus.AUTHENTICATED

```

FILE: state/card_inserted_state.py (43 lines)

```

1| from __future__ import annotations
2|
3| from typing import TYPE_CHECKING
4|
5| from ..enums.atm_status import ATMStatus
6| from ..model.card import Card
7| from .atm_state import ATMState
8|
9| if TYPE_CHECKING:
10|     from ..service.atm_machine import ATMMachine
11|
12|
13| class CardInsertedState(ATMState):
14|     def __init__(self, atm_machine: ATMMachine):
15|         self._atm_machine = atm_machine
16|
17|     def insert_card(self, card: Card) -> None:
18|         print("Card already inserted.")
19|
20|     def enter_pin(self, pin: str) -> None:
21|         from .authenticated_state import AuthenticatedState
22|
23|         if self._atm_machine.current_card.pin == pin:
24|             print("PIN correct. Authenticated.")
25|             self._atm_machine.set_state(AuthenticatedState(self._atm_machine))
26|         else:
27|             print("Invalid PIN.")
28|
29|     def select_option(self, option: str) -> None:
30|         print("Enter PIN first.")
31|
32|     def dispense_cash(self, amount: int) -> None:
33|         print("Enter PIN before dispensing.")
34|
35|     def eject_card(self) -> None:
36|         from .idle_state import IdleState
37|
38|         self._atm_machine.current_card = None
39|         print("Card ejected.")
40|         self._atm_machine.set_state(IdleState(self._atm_machine))
41|
42|     def get_status(self) -> ATMStatus:
43|         return ATMStatus.CARD_INSERTED

```

FILE: state/dispense_cash_state.py (65 lines)

(cont.) state/dispense_cash_state.py

```

1| from __future__ import annotations
2|
3| from typing import TYPE_CHECKING
4|
5| from ..cor.cash_dispenser import CashDispenser
6| from ..cor.cash_dispenser_chain_builder import CashDispenserChainBuilder
7| from ..enums.atm_status import ATMStatus
8| from ..model.card import Card
9| from .atm_state import ATMState
10|
11| if TYPE_CHECKING:
12|     from ..service.atm_machine import ATMMachine
13|
14|
15| class DispenseCashState(ATMState):
16|     def __init__(self, atm_machine: ATMMachine):
17|         self._atm_machine = atm_machine
18|         self._chain: CashDispenser = CashDispenserChainBuilder.build_chain()
19|
20|     def insert_card(self, card: Card) -> None:
21|         print("Transaction in progress. Cannot insert another card.")
22|
23|     def enter_pin(self, pin: str) -> None:
24|         print("Already authenticated.")
25|
26|     def select_option(self, option: str) -> None:
27|         print("Option already selected.")
28|
29|     def dispense_cash(self, amount: int) -> None:
30|         atm_balance = self._atm_machine.atm.cash_available
31|         account_balance = self._atm_machine.current_card.account.balance
32|
33|         if amount > atm_balance:
34|             print(f"ATM has insufficient cash. Cannot dispense {amount}")
35|             self.eject_card()
36|             return
37|
38|         if amount > account_balance:
39|             print("Insufficient account balance.")
40|             self.eject_card()
41|             return
42|
43|         # Now check if note combination is possible
44|         if self._chain.can_dispense(self._atm_machine.atm, amount):
45|             self._chain.dispense(self._atm_machine.atm, amount)
46|
47|         # Deduct from ATM cash & account balance
48|         self._atm_machine.atm.cash_available = atm_balance - amount
49|         self._atm_machine.current_card.account.balance = account_balance - amount
50|
51|         self.eject_card()
52|         print(f"Cash dispensed: {amount}")
53|     else:
54|         print("Cannot dispense requested amount with available denominations.")
55|         self.eject_card()
56|
57|     def eject_card(self) -> None:
58|         from .idle_state import IdleState
59|
60|         self._atm_machine.current_card = None
61|         print("Card ejected.")
62|         self._atm_machine.set_state(IdleState(self._atm_machine)) # use factory
63|
64|     def get_status(self) -> ATMStatus:
65|         return ATMStatus.DISPENSE_CASH

```

FILE: state/idle_state.py (37 lines)

```

1| from __future__ import annotations
2|
3| from typing import TYPE_CHECKING
4|
5| from ..enums.atm_status import ATMStatus
6| from ..model.card import Card
7| from .atm_state import ATMState
8|
9| if TYPE_CHECKING:
10|     from ..service.atm_machine import ATMMachine
11|
12|
13| class IdleState(ATMState):
14|     def __init__(self, atm_machine: ATMMachine):
15|         self._atm_machine = atm_machine
16|
17|     def insert_card(self, card: Card) -> None:
18|         from .card_inserted_state import CardInsertedState
19|
20|         self._atm_machine.current_card = card
21|         print("Card inserted.")
22|         self._atm_machine.set_state(CardInsertedState(self._atm_machine))
23|
24|     def enter_pin(self, pin: str) -> None:
25|         print("No card inserted.")
26|
27|     def select_option(self, option: str) -> None:
28|         print("No card inserted.")
29|
30|     def dispense_cash(self, amount: int) -> None:
31|         print("No card inserted.")

```



```
32|
33| def eject_card(self) -> None:
34|     print("No card to eject.")
35|
36| def get_status(self) -> ATMStatus:
37|     return ATMStatus.IDLE
```

FILE: factory/atm_state_factory.py (25 lines)

```
1| from typing import TYPE_CHECKING
2|
3| from ..enums.atm_status import ATMStatus
4| from ..state.atm_state import ATMState
5| from ..state.authenticated_state import AuthenticatedState
6| from ..state.card_inserted_state import CardInsertedState
7| from ..state.dispense_cash_state import DispenseCashState
8| from ..state.idle_state import IdleState
9|
10| if TYPE_CHECKING:
11|     from ..service.atm_machine import ATMMachine
12|
13|
14| class ATMStateFactory:
15|     @staticmethod
16|     def get_state(status: ATMStatus, machine: "ATMMachine") -> ATMState:
17|         if status == ATMStatus.IDLE:
18|             return IdleState(machine)
19|         if status == ATMStatus.CARD_INSERTED:
20|             return CardInsertedState(machine)
21|         if status == ATMStatus.AUTHENTICATED:
22|             return AuthenticatedState(machine)
23|         if status == ATMStatus.DISPENSE_CASH:
24|             return DispenseCashState(machine)
25|         raise ValueError(f"Unknown ATM status: {status}")
```

FILE: repository/atm_repository.py (18 lines)

```
1| from typing import Optional
2|
3| from ..enums.atm_status import ATMStatus
4| from ..model.atm import ATM
5|
6|
7| class ATMRepository:
8|     def __init__(self):
9|         self._atms: dict[str, ATM] = {}
10|
11|     def save(self, atm: ATM) -> None:
12|         self._atms[atm.id] = atm
13|
14|     def get_by_id(self, id: str) -> Optional[ATM]:
15|         return self._atms.get(id)
16|
17|     def update_atm_status_by_id(self, id: str, new_status: ATMStatus) -> None:
18|         self._atms[id].status = new_status
```

FILE: enums/atm_status.py (8 lines)

```
1| from enum import Enum
2|
3|
4| class ATMStatus(Enum):
5|     IDLE = "IDLE"
6|     CARD_INSERTED = "CARD_INSERTED"
7|     AUTHENTICATED = "AUTHENTICATED"
8|     DISPENSE_CASH = "DISPENSE_CASH"
```